

# Rust – Chapitre 9 : Gestion des erreurs

1. [Les deux catégories d'erreurs](#)
2. [Erreurs irrécupérables avec `panic!`](#)
3. [La trace d'appels \(\*backtrace\*\)](#)
4. [Erreurs récupérables avec `Result`](#)
5. [Gérer un `Result` avec `match`](#)
6. [Filtrer sur différentes erreurs](#)
7. [Raccourcis : `unwrap` et `expect`](#)
8. [Propagation des erreurs](#)
9. [L'opérateur `?`](#)
10. [Où utiliser `?`](#)
11. [`panic!` ou `Result` : comment choisir ?](#)
12. [Types personnalisés pour la validation](#)

## 1 - Les deux catégories d'erreurs

`Rust` distingue deux grandes familles d'erreurs :

| Catégorie            | Mécanisme                       | Quand l'utiliser                                      |
|----------------------|---------------------------------|-------------------------------------------------------|
| <b>Irrécupérable</b> | <code>panic!</code>             | Bogue, état invalide, invariant violé                 |
| <b>Récupérable</b>   | <code>Result&lt;T, E&gt;</code> | Échec prévisible (fichier absent, entrée invalide...) |

## 2 - Erreurs irrécupérables avec `panic!`

La macro `panic!` arrête immédiatement l'exécution du programme et affiche un message d'erreur.

```
// Exemple : tentative d'accès à un index invalide
fn obtenir_livre(bibliotheque: &Vec<String>, index: usize) -> &String {
    if index >= bibliotheque.len() {
        panic!(
            "Index {} hors limites : la bibliothèque ne contient que {} livre(s).",
            index,
            bibliotheque.len()
        );
    }
    &bibliotheque[index]
}
```

## Ce que Rust affiche en cas de panique :

```
thread 'main' panicked at src/main.rs:4:5:  
Index 99 hors limites : la bibliothèque ne contient que 3 livre(s).  
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

### ✍ À retenir

`panic!` est adapté aux situations où continuer serait dangereux ou absurde. Ce n'est pas un mécanisme de gestion d'erreurs ordinaires.

## 3 - La trace d'appels (*backtrace*)

La trace d'appels liste toutes les fonctions appelées jusqu'au point de panique.

### 3.1 - Activer la trace d'appels

```
RUST_BACKTRACE=1 cargo run
```

### 3.2 - Lire la trace d'appels

- Lire depuis le haut jusqu'à voir un fichier que nous avons nous-même écrit.
- Les lignes au-dessus : code que le code a appelé (bibliothèques, std...).
- Les lignes en dessous : code qui a été appelé par le code écrit par nous-même.
- C'est le fichier écrit qui indique l'origine du problème.

### ✍ Prérequis

Les symboles de débogage doivent être activés (c'est le cas par défaut avec `cargo run` ou `cargo build` sans `--release`).

## 4 - Erreurs récupérables avec `Result`

L'`enum Result<T, E>` est définie ainsi dans la bibliothèque standard :

```
enum Result<T, E> {  
    Ok(T),    // succès : contient la valeur de type T  
    Err(E),   // échec  : contient l'erreur de type E  
}
```

`Result` est importé automatiquement par le prélude : pas besoin d'écrire `Result::Ok` ou `Result::Err`.

```
// Exemple : ouvrir un fichier
use std::fs::File;

fn main() {
    let resultat = File::open("bibliotheque.json");
    // résultat est de type Result<File, io::Error>
}
```

## 5 - Gérer un `Result` avec `match`

```
use std::fs::File;

fn main() {
    let rsultat = File::open("bibliotheque.json");

    let fichier = match resultat {
        Ok(fichier) => fichier,
        Err(erreur) => panic!(
            "Impossible d'ouvrir le fichier de bibliothèque : {erreur:?}"
        ),
    };
}
```

## 6 - Filtrer sur différentes erreurs

On peut imbriquer des `match` pour réagir différemment selon le type d'erreur :

```
use std::fs::File;
use std::io::ErrorKind;

fn main() {
    let resultat = File::open("bibliotheque.json");

    let fichier = match resultat {
        Ok(fichier) => fichier,
        Err(erreur) => match erreur.kind() {
            // Le fichier n'existe pas : on le crée
            ErrorKind::NotFound => match File::create("bibliotheque.json") {
                Ok(nouveau_fichier) => nouveau_fichier,
                Err(e) => panic!("Impossible de créer le fichier : {e:?}"),
            },
            // Autre erreur (permissions, etc.)
            _ => panic!("Erreur lors de l'ouverture du fichier : {erreur:?}"),
        },
    };
}
```

## 6.1 - Version plus concise avec `unwrap_or_else` et des closures

```
use std::fs::File;
use std::io::ErrorKind;

fn main() {
    let fichier = File::open("bibliotheque.json").unwrap_or_else(|erreur| {
        if erreur.kind() == ErrorKind::NotFound {
            File::create("bibliotheque.json").unwrap_or_else(|erreur| {
                panic!("Impossible de créer le fichier : {erreur:?}");
            })
        } else {
            panic!("Erreur lors de l'ouverture du fichier : {erreur:?}");
        }
    });
}
```

### `unwrap_or_else`

- si `Ok(v)`, retourne `v`
- si `Err(e)`, exécute la closure avec `e`.

## 7 - Raccourcis : `unwrap` et `expect`

### 7.1 - `unwrap`

Retourne la valeur si `Ok`, appelle `panic!` automatiquement si `Err` :

```
// Pratique pour les prototypes ou quand on est certain du succès
let fichier = File::open("bibliotheque.json").unwrap();
```

### 7.2 - `expect`

Identique à `unwrap`, mais permet de personnaliser le message de panique :

```
let fichier = File::open("bibliotheque.json")
    .expect("Le fichier bibliotheque.json devrait exister dans ce projet");
```

| Méthode             | Message de panique            | Usage recommandé          |
|---------------------|-------------------------------|---------------------------|
| <code>unwrap</code> | Message générique automatique | Prototypage rapide        |
| <code>expect</code> | Message personnalisé          | Code de production, tests |

### Conseil

En production, préférer `expect` avec un message explicatif. Si les hypothèses s'avèrent fausses un jour, nous aurons un contexte précieux pour déboguer.

## 8 - Propagation des erreurs

Au lieu de gérer l'erreur dans la fonction, on la remonte au code appelant :

```
use std::fs::File;
use std::io::{self, Read};

fn lire_catalogue_depuis_fichier() → Result<String, io::Error> {
    let resultat_fichier = File::open("bibliotheque.json");

    let mut fichier = match resultat_fichier {
        Ok(fichier) ⇒ fichier,
        Err(e) ⇒ return Err(e), // retour anticipé : on propage l'erreur
    };

    let mut contenu = String::new();

    match fichier.read_to_string(&mut contenu) {
        Ok(_) ⇒ Ok(contenu), // succès : on retourne le contenu
        Err(e) ⇒ Err(e), // échec : on propage l'erreur
    }
}
```

**Types de retour :** `Result<String, io::Error>`

- `Ok(String)` → le catalogue lu depuis le fichier
- `Err(io::Error)` → l'erreur remontée au code appelant

### Pourquoi propager ?

Le code appelant a souvent plus de contexte pour décider quoi faire d'une erreur (paniquer, utiliser une valeur par défaut, réessayer...).

## 9 - L'opérateur ?

L'opérateur `?` est un raccourci élégant pour la propagation d'erreurs. Il remplace le `match` de retour anticipé.

### 9.1 - Comportement

- Si `Ok(v)` → retourne `v` et continue
- Si `Err(e)` → convertit `e` via le trait `From`, puis retourne `Err(e)` de la fonction

```
use std::fs::File;
use std::io::{self, Read};

// Version avec ? – équivalente au listing précédent
fn lire_catalogue_depuis_fichier() → Result<String, io::Error> {
    let mut fichier = File::open("bibliotheque.json"?);
    let mut contenu = String::new();
    fichier.read_to_string(&mut contenu)?;
    Ok(contenu)
}
```

## 9.2 - Version encore plus concise (chaînage)

```
fn lire_catalogue_depuis_fichier() → Result<String, io::Error> {
    let mut contenu = String::new();
    File::open("bibliotheque.json")?.read_to_string(&mut contenu)?;
    Ok(contenu)
}
```

## 9.3 - Version la plus courte (avec `fs::read_to_string`)

```
use std::fs;
use std::io;

fn lire_catalogue_depuis_fichier() → Result<String, io::Error> {
    fs::read_to_string("bibliotheque.json")
}
```

### Différence avec `match`

`?` appelle automatiquement la fonction `from` du trait `From` pour convertir le type d'erreur vers celui déclaré en retour de la fonction.

## 10 - Où utiliser `?`

`?` ne peut être utilisé que dans une fonction dont le type de retour est `Result<T, E>`, `Option<T>`, ou un type implémentant `FromResidual`.

### 10.1 - Erreur classique : utiliser `?` dans `main()` par défaut

```
// ✗ Ne compile pas : main() retourne ()
use std::fs::File;

fn main() {
```

```
let fichier = File::open("bibliotheque.json");
}
```

## 10.2 Solution : modifier le type de retour de `main()`

```
// ✓ Compile : main() retourne Result
use std::error::Error;
use std::fs::File;

fn main() → Result<(), Box<dyn Error>> {
    let fichier = File::open("bibliotheque.json");
    Ok(())
}
```

## 10.3 ? sur `Option<T>` :

```
// Trouver le premier auteur d'un catalogue textuel
fn premier_auteur(catalogue: &str) → Option<&str> {
    catalogue.lines().next()?.split(':').next()
}
```

- Si `None` → retourne `None` prématurément
- Si `Some(v)` → extrait `v` et continue

### Règle

On ne peut pas mélanger ? sur `Result` et ? sur `Option` dans la même fonction. Utiliser `.ok()` (`Result` → `Option`) ou `.ok_or(e)` (`Option` → `Result`) pour convertir.

## 11 - `panic!` ou `Result` : comment choisir ?

### 11.1 - Utiliser `panic!` quand...

- L'état du programme est fondamentalement invalide (invariant cassé, bogue du développeur).
- Continuer serait dangereux (accès mémoire hors limites, données corrompues).
- Quand on appelle du code externe qui retourne un état invalide que nous ne pouvons pas corriger.
- Quand nous produisons du code d'exemple, du code prototype ou un test.

### 11.2 - Utiliser `Result` quand...

- L'échec est prévisible et attendu (fichier absent, réseau indisponible, données malformées).
- Le code appelant doit pouvoir décider comment réagir.
- Quand nous laissons l'utilisateur choisir.

### 11.3 - Dans les exemples, prototypes et tests

```
// Dans un prototype : unwrap est acceptable
let catalogue = fs::read_to_string("bibliotheque.json").unwrap();

// Dans un test : expect est préférable
let catalogue = fs::read_to_string("bibliotheque.json")
    .expect("Le fichier de test devrait exister");
```

## 11.4 - Quand nous en savons plus que le compilateur

```
use std::net::IpAddr;

// On sait que cette adresse est valide – expect est justifié
let serveur: IpAddr = "127.0.0.1"
    .parse()
    .expect("L'adresse IP du serveur local est toujours valide");
```

## 12 - Types personnalisés pour la validation

Plutôt que de répéter des vérifications partout, on peut encoder les contraintes dans le système de types en créant un type dédié.

```
// Exemple : une année de publication valide (entre 1492 et 2026)

pub struct AnneePublication {
    valeur: u32,
}

impl AnneePublication {
    /// Crée une AnneePublication valide.
    ///
    /// # Paniques
    /// Panique si `valeur` n'est pas comprise entre 1492 et 2026.
    pub fn new(valeur: u32) → AnneePublication {
        if valeur < 1492 || valeur > 2026 {
            panic!(
                "L'année de publication doit être entre 1492 et 2026, reçu {}.",
                valeur
            );
        }
        AnneePublication { valeur }
    }

    // Accesseur (getter) – le champ `valeur` étant privé,
    // c'est le seul moyen d'y accéder depuis l'extérieur
    pub fn valeur(&self) → u32 {
        self.valeur
    }
}

// Utilisation : on est certain que l'année est valide dès la création
fn afficher_livre(titre: &str, annee: AnneePublication) {
```



```
println!("{}", titre, annee.valeur());  
}
```

### Pourquoi le champ `valeur` est-il privé ?

Si `valeur` était public, n'importe quel code pourrait écrire `AnneePublication { valeur: 9999 }` en contournant la validation. En le rendant privé, la seule façon de créer une `AnneePublication` est de passer par `new`, qui garantit la validité.

#### Avantage

Les fonctions qui acceptent ou retournent une `AnneePublication` n'ont plus besoin de re-valider : le type lui-même est une garantie.

## 13 - Récapitulatif visuel

